Reinforcement learning from human feedback, for the very curious

How a language model learns to be good — and why the mathematics of "good" turns out to be a single subtraction

This essay is part of a series written in what we call a *mnemonic medium*. Along the way you will find small boxes containing a question and its answer. Those boxes are not decoration. They are the essay testing you — and if you take them seriously, you will remember this material for years instead of days.

Here is how to use them. When a box appears, **cover the answer with your hand**, say your answer out loud, and only then read what is written. If you get it wrong, that is useful: the effort of a failed recall is what makes the next one stick. If you get it right, move on. Each box is numbered so you can find it again; every question and answer is also collected in the Appendix, and in a plain-text file you can import into any spaced-repetition system.

One more thing before we begin. This essay is built from two sources: a set of handwritten lecture notes (pages 128–146 of an IIT Madras generative-AI course), and a recording of the professor delivering them. Where the notes and the lecture disagree — and they do, in several places, including two outright errors — we will stop and look at it. You will find those moments marked *A note on the notes*. Learning to distrust a source is part of learning.

PART I — THE PROBLEM WITH A VERY GOOD PARROT

1 · A machine that finishes your sentence 2 · The thing we cannot write down 3 · Two kinds of learning from data 4 · The reward you can check

PART II — AN AGENT IN A WORLD

5 · A joystick and a score 6 · The six things that make a game 7 · One number for an entire life 8 · The distribution over futures 9 · The integral we cannot do 10 · The trick that fools a derivative 11 · The gift 12 · Too noisy to be useful 13 · Did this action help? 14 · The equation on every page

PART III — PUTTING A LANGUAGE MODEL INSIDE THE MACHINE

15 · A prompt is a state 16 · Stale samples, and the ratio that fixes them 17 · When the ratio explodes 18 · The clip 19 · What the clip does, in numbers

PART IV — WHERE THE NUMBER COMES FROM

20 · You cannot grade on an absolute scale 21 · Two answers, one question 22 · A reward defined only by its differences 23 · Goodhart, with a gradient

PART V — THE CANCELLATION

24 · An objective with a closed-form answer 25 · The function that cancels 26 · DPO 27 · Removing the value function

PART VI — WHAT WE GLOSSED OVER

28 · Five places the notes are wrong 29 · What this essay does not tell you Appendix · Every question in this essay

PART I — THE PROBLEM WITH A VERY GOOD PARROT

1 · A machine that finishes your sentence

Suppose you train a model on a very large pile of text. You give it one job: given everything written so far, say what comes next. Do that a few trillion times and something strange happens. The model gets good enough that it can finish a sentence, then a paragraph, then a proof sketch, then a program.

Formally, it has learned a conditional distribution over the next token. Call the parameters θ, the vocabulary $\mathcal{V}$, and the situation at step t the state s. Then what the model gives you is this:

$$\pi_\theta(a \mid s) \;=\; \Pr\left(x_t = v \mid x_{<t}\right), \qquad v \in \mathcal{V}$$

Read the left-hand side carefully, because it is the strangest thing in this essay. We have written the model's output as a *policy*: a distribution over actions given a situation. That is a word from a different field entirely, and the whole rest of this essay is about why it is the right word.

Because here is the trouble. A model trained this way is an astonishing imitator and a poor judge. It will continue a sentence the way the internet would continue it — including the parts of the internet that are cruel, or wrong, or simply unhelpful. It has no term in its objective that says *this completion is better than that one*. It only knows what tends to follow what.

QUESTION 1.1

What exactly does a next-token-prediction model learn — a program, a database of answers, or something else?

ANSWER

A conditional probability distribution over the next token given the previous tokens: $\pi_\theta(a \mid s)$. Everything else it appears to do is sampling from that distribution.

QUESTION 1.2

Why doesn't that training objective already make the model helpful?

ANSWER

Because it only rewards *matching the corpus*. There is no term in the objective that compares two possible responses and says which is better. The corpus contains cruelty and error as faithfully as it contains kindness and truth, and the model learns to reproduce all of it.

2 · The thing we cannot write down

So we want the model to be *good*. The lecture we are following states the goal plainly: to make the model compatible with a desired, pre-defined rule or behaviour, as specified by human annotations or by fixed rules.

Why not simply collect examples of good behaviour and fine-tune on them? People do this, and it helps. But it runs into two walls, and it is worth naming them precisely, because they explain everything that follows.

The first wall is about **format**. A person is quite good at looking at two answers and saying "the second one is better." A person is much worse at *writing* the ideal answer from scratch. If you ask for demonstrations you get a little data of high quality. If you ask for comparisons you get a lot of data of decent quality. Comparisons are the only label that scales — but a comparison is not a target string, so there is nothing to put through a cross-entropy loss.

The second wall is about **location**. Whether a response is helpful is not a property of any single token. It is a property of the whole response — a derivation is correct or it is not; a refusal is appropriate or it is not. You cannot label token number 47 as "good". The judgement lives at the end, and it is about the entire thing.

Put those together. We need to optimise a number that arrives once, at the end, judging a whole sequence. That is not a supervised-learning problem. It is a problem about *acting*, and it has a name.

QUESTION 2.1

What is the "format" wall that stops us from just using supervised fine-tuning?

ANSWER

Human preference data comes as *comparisons*, not as demonstrations. A ranking is not a target string, so there is nothing to compute a per-token loss against. Comparisons are cheap and scalable; ideal answers are neither.

QUESTION 2.2

What is the "location" wall?

ANSWER

The quality signal is a property of the *whole sequence*, not of individual tokens. It arrives once, at the end. You cannot assign a per-token label to whether a derivation is correct.

3 · Two kinds of learning from data

It helps to place this next to what you already know. There are three ways a machine can learn from data, and they differ in what they are handed.

$$\begin{aligned} \text{supervised:} \quad & (x, y) \sim p(x, y) && \Longrightarrow \ \text{learn } p(y \mid x) \\ \text{unsupervised:} \quad & x \sim p(x) && \Longrightarrow \ \text{learn } p(x) \\ \text{reinforcement:} \quad & \text{no dataset} && \Longrightarrow \ \text{learn } \pi_\theta(a \mid s) \end{aligned}$$

Supervised learning *imitates a label*. Reinforcement learning *optimises a scalar obtained by acting*. Notice that the third row has a hole where the data used to be. There is no dataset of correct actions. Instead there is an environment which, when you do something, tells you what it was worth.

That is a strange trade. You give up the rich per-example supervision of a label, and in exchange you get something a label can never give you: the ability to be judged on outcomes you were never shown.

QUESTION 3.1

In one line: what is the difference between supervised learning and reinforcement learning?

ANSWER

Supervised learning imitates a label; reinforcement learning optimises a scalar reward obtained by acting in an environment. The second has no dataset of correct actions — only feedback.

4 · The reward you can check

Here is the single most practical idea in this whole subject, and the handwritten notes never mention it. The professor says it in the first three minutes.

Reinforcement learning works *best* when the reward is **verifiable** — when you can check it, mechanically, with no human in the loop. Did the program pass the unit test? Does the answer equal 42? Is the proof machine-checkable? Those are yes-or-no questions with a definite answer.

And now look at which tasks modern language models are best at. Mathematics. Programming. Anything with a checker. This is not a coincidence. In those domains the reward is essentially a coin: $r \in \{0, 1\}$, free to compute, noiseless, and available in unlimited quantity. You can run the model a million times and grade every attempt.

Compare a chatbot. What is the reward for "was this reply kind, accurate and appropriately brief"? It is a judgement, not a check. Two annotators give the same reply a 6 and a 9. There is no test case. So you cannot compute the reward — you have to *learn* a function that predicts it, from human comparisons. And every error in that learned function becomes an error in your model.

The professor puts the consequence in one sentence, and it is worth memorising verbatim: *your policy gradient algorithm is as good as your reward model.*

QUESTION 4.1

What makes a reward "verifiable", and give the two domains where this works best.

ANSWER

Verifiable means deterministic and mechanically checkable — a test case passes or it doesn't. Mathematics and programming are the two domains, because a checker can grade the outcome without a human, for free, at unlimited scale.

QUESTION 4.2

Why is an open-ended chat assistant the hard case?

ANSWER

Because there is no checker. The reward for "was this reply good" is a human judgement, so it must be *learned* from preference comparisons rather than computed — and the learned reward model becomes the ceiling on how good the assistant can become.

A NOTE ON THE NOTES

The lecture's one-line summary — "your policy gradient algorithm is as good as your reward model" — appears nowhere on pages 128–146. Nor does the word *verifiable*. If you revise from the notes alone you will learn the machinery of PPO and miss the reason it is used where it is used.

PART II — AN AGENT IN A WORLD

5 · A joystick and a score

Before there were language models, the standard testbed for this kind of learning was a video game. The professor, who is old enough to remember them, apologises to his students for the reference. We will not.

Picture an Atari machine. There is a screen. There is a joystick with a handful of moves: left, right, jump. There is a score in the corner. You see the screen, you move the joystick, the screen changes, and occasionally the score goes up.

Now ask: what would it take for a program to learn to play this, with no instructions, no manual, and no teacher — only the screen and the score?

Notice what information the program has. At each moment it is in some *situation* — the position of things on the screen. It chooses an *action* from a small menu. The machine then decides what the next situation is, and pays out a number. That is all. The program never learns the rules of the game. It never reads the manual. It only ever learns: *in situations like this, that action tends to lead to more points.*

Hold on to that last sentence, because in a few pages we will prove something surprising about it: the program can learn to play well **without ever knowing how the machine decides what happens next**. That is not an approximation. It is exactly true, and it is the reason this machinery works on language models, where nobody knows the "rules" at all.

QUESTION 5.1

In the Atari framing, what is the "state", what is the "action", and where does the "reward" come from?

ANSWER

The state is the current screen (or the things on it), the action is one of the joystick moves, and the reward is the score. The environment — the machine — supplies the next state and the reward.

6 · The six things that make a game

That description can be made precise, and once it is precise you can prove things about it. Six objects are enough.

SYMBOL	WHAT IT IS	IN THE ATARI GAME
$\mathcal{S}$	state space	all the screens you could be looking at
$\mathcal{A}$	action space	the joystick's moves
$p(s' \mid s, a)$	transition kernel	how the machine reacts to your move
$r(s, a)$	reward	the score change, a single number
γ	discount factor	how much you care about the future
$\rho_0(s_0)$	initial state distribution	which screen the game starts on

Add one more object, which is the thing we are actually trying to learn: a *policy* $\pi_\theta(a \mid s)$, a distribution over actions given the current state. The policy is the entire behaviour of the agent. In the lecture, the professor is emphatic that "agent" and "policy" are the same word: if there is an agent there is a policy, because that is what defines an agent.

The word *Markov* in "Markov decision process" is doing exactly one job. It says the transition kernel depends only on the current state and action, never on how you got there. The future is conditionally independent of the past, given the present. That is why the state must contain everything relevant — in our language model, the state will have to be the entire text so far, not just the last token.

QUESTION 6.1

What does the word "Markov" assert, and what does that force the state to be?

ANSWER

It asserts that the transition kernel depends only on the current state and action: $p(s_{t+1} \mid s_t, a_t, \dots, s_0) = p(s_{t+1} \mid s_t, a_t)$. So the state must contain everything about the past that matters — for a language model, the whole text so far.

7 · One number for an entire life

The agent does not receive one reward. It receives a sequence of them, one per step, and we need to collapse that sequence into a single number we can maximise. The standard way is to add them up — but with a twist.

$$R(\tau) \;=\; \sum_{t=0}^{T} \gamma^t \, r(s_t, a_t), \qquad 0 < \gamma < 1$$

Winding the sum by γ^t *discounts the future*. A point now is worth more than a point a hundred steps from now. Two reasons are usually given, and they are worth separating.

The engineering reason: if the game never ends, the raw sum may be infinite, and no finite sum has a maximum. Discounting makes the total finite and the problem well-posed. The behavioural reason: it is a dial for how far ahead the agent bothers to look. As $\gamma \to 0$ it becomes greedy and only sees the next step; as $\gamma \to 1$ it becomes patient and weights the whole future equally.

> A NOTE ON THE NOTES
>
> Two small distortions to avoid. Page 130 writes the return as $\sum_t r(s_t, a_t)$ with *no* γ — even though γ was declared one page earlier. And the lecture, describing what γ is for, says it makes "recent" rewards matter most. Mechanically it is the reverse framing: γ^t makes *distant* rewards matter *less*. The mechanism is exponential decay in time, not recency.

QUESTION 7.1

What are the two distinct reasons for the discount factor γ?

ANSWER

(i) *Well-posedness*: on an infinite horizon the undiscounted sum may diverge, so no finite maximum exists. (ii) *Horizon control*: γ dials how far ahead the agent optimises.

8 · The distribution over futures

Now a change of view that makes the mathematics possible.

Run the agent once. You get a film: the starting screen, the first action, the next screen, the next action, and so on to the end. Call that whole film a *trajectory* τ. The agent does not produce a single action; it produces a distribution over films.

And because the world is a Markov decision process, that distribution factorises beautifully. Start in a state drawn from ρ_0. Then at each step multiply the probability the policy chose that action, and the probability the world responded with that next state:

$$p_\theta(\tau) \;=\; \rho_0(s_0) \prod_{t=0}^{T} \pi_\theta(a_t \mid s_t)\; p(s_{t+1} \mid s_t, a_t)$$

This is nothing but the chain rule of probability, applied to a process that unrolls in time. The lecture says exactly that: "it's just the chain rule".

Now look at that product with your eyes narrowed. There are three kinds of term. One is the starting distribution. One is the world's rulebook. One is your policy. **Only one of the three contains θ.**

That single observation is the hinge on which the entire subject turns. Everything that follows — every algorithm, every paper — exploits it. Remember it.

QUESTION 8.1

Write the trajectory distribution, and say which factors depend on θ.

ANSWER

$p_\theta(\tau) = \rho_0(s_0) \prod_t \pi_\theta(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t)$. Only the *policy* factors π_θ depend on θ. The initial distribution and the transition kernel are properties of the world.

QUESTION 8.2

Why does that factorisation matter so much?

ANSWER

Because when we differentiate with respect to θ, the two θ-free factors differentiate to zero and vanish. The gradient will depend only on the policy — and never on the world's transition rule.

9 · The integral we cannot do

We now have an objective. Maximise the expected return over trajectories drawn from your own policy:

$$J(\theta) \;=\; \mathbb{E}_{\tau \sim p_\theta}\big[R(\tau)\big] \;=\; \int p_\theta(\tau)\, R(\tau)\, d\tau$$

And we want to climb it: compute $\nabla_\theta J$ and step uphill.

So let us differentiate. The gradient slides inside the integral:

$$\nabla_\theta J(\theta) \;=\; \int \underbrace{\nabla_\theta p_\theta(\tau)}_{\text{the measure}}\, R(\tau)\, d\tau$$

Now stop and look at where θ is. It is not inside the reward — the reward is handed to you by the world and has no parameters. θ is inside the *distribution*. You are not differentiating a function of θ. You are differentiating a distribution over θ, and integrating the result against a fixed function.

This is the crux, so let us say the wrong thing and see why it fails. You might try to compute $\mathbb{E}[\nabla_\theta R(\tau)]$. But $R(\tau)$ does not contain θ, so that gradient is exactly zero, for every trajectory, always. It tells you nothing. The only way the objective changes when you move θ is by changing *which trajectories happen*.

And we cannot even do that integral by brute force. The set of possible trajectories has size roughly $|\mathcal{V}|^T$ — for a language model, a vocabulary of tens of thousands raised to the power of a few hundred tokens. You could not enumerate it before the heat death of the universe. And p_θ itself is not something you have a formula for; you have a sampler.

This is the same shape of problem you meet in variational inference: an intractable integral over a distribution you cannot write down. And it is solved by the same kind of move — get θ inside a logarithm, then estimate by sampling.

QUESTION 9.1

Why is $\mathbb{E}_\tau\big[\nabla_\theta R(\tau)\big]$ useless?

ANSWER

Because $R(\tau)$ contains no θ, so its gradient is identically zero. The parameters influence the objective only by changing the *distribution over trajectories*, never the reward of a given trajectory.

QUESTION 9.2

In one phrase: what is being differentiated in $\nabla_\theta J$?

ANSWER

The *measure* — the probability distribution over trajectories — not the integrand.

10 · The trick that fools a derivative

Here is a two-line identity that you have probably seen and possibly dismissed as a curiosity. It is the whole game.

$$\nabla_\theta p_\theta \;=\; p_\theta \cdot \nabla_\theta \log p_\theta$$

Why is it true? Because $\nabla \log p = (1/p)\,\nabla p$, so multiplying by p gives back ∇p. That is all. A function times one over itself times its derivative is its derivative.

Watch what it buys. Substitute it into the integral:

$$\begin{aligned}
\nabla_\theta J(\theta) &= \int \nabla_\theta p_\theta(\tau)\, R(\tau)\, d\tau && \text{(start)} \\
&= \int p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, R(\tau)\, d\tau && \text{(the trick)} \\
&= \mathbb{E}_{\tau\sim p_\theta}\Big[\nabla_\theta \log p_\theta(\tau)\, R(\tau)\Big] && \text{(an expectation!)}
\end{aligned}$$

Look at what changed. The left side was an integral over an object we cannot even write down. The right side is an *expectation* — which means we can estimate it by sampling

trajectories and averaging. We have converted an impossible integral into a thing you can do with a for-loop.

And the quantity inside is computable, because of the factorisation from chapter 8. Take the log of the product: the log of a product is a sum of logs. Differentiate: the two θ-free terms die.

$$\log p_\theta(\tau) = \underbrace{\log \rho_0(s_0)}_{\text{no } \theta} + \sum_t \underbrace{\log \pi_\theta(a_t \mid s_t)}_{\text{has } \theta} + \sum_t \underbrace{\log p(s_{t+1} \mid s_t, a_t)}_{\text{no } \theta}$$

$$\implies \quad \nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

Three terms go in; one comes out. The world's rulebook has been differentiated away.

QUESTION 10.1

State the log-derivative trick and prove it in one line.

ANSWER

$\nabla_\theta p_\theta = p_\theta \, \nabla_\theta \log p_\theta$. Proof: $\nabla \log p = (1/p)\nabla p$, so $p \cdot (1/p)\nabla p = \nabla p$.

QUESTION 10.2

What does the trick convert the problem *into*, and why does that help?

ANSWER

It converts the gradient of an intractable integral into an *expectation* of a computable quantity, $\mathbb{E}[\nabla_\theta \log p_\theta(\tau) R(\tau)]$, which can be estimated by sampling trajectories and averaging.

11 · The gift

Put the pieces together and something remarkable falls out.

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\Big[\Big(\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big) R(\tau)\Big]}$$

Read the right-hand side and list what it needs. It needs the policy, which you have — it is your model. It needs to be able to compute the log-probability of a token, which is one forward pass. It needs trajectories, which you get by sampling. And that is all.

Now list what it **does not** need. It does not need $p(s_{t+1} \mid s_t, a_t)$ — the world's transition rule. Not approximately, not estimated: the term is simply gone.

Sit with how strange that is. In the Atari game, the “rules” are the game's code, which you could in principle read. In a conversation, the transition rule is *how the world responds*

to what your model just said — the laws of physics, the state of mind of the person reading, the state of the internet. Nobody has that written down. Nobody ever will.

The policy gradient theorem says you never need it. You need only the ability to try things and be told how it went. The professor calls this a very, very strong result. It is the reason this machinery exists at all.

QUESTION 11.1

What does the policy gradient theorem free you from needing, and why is that the difference between theory and practice here?

ANSWER

It removes any dependence on the state-transition distribution $p(s' \mid s, a)$. For a language model interacting with the world that rule is unknowable — so without this result the whole approach would be unusable.

QUESTION 11.2

Name the three things you need in order to run this estimator.

ANSWER

(i) $\nabla_\theta \log \pi_\theta$, free from backprop through the model; (ii) an estimate of the advantage $\hat{A}$; (iii) many rollouts — batches of sampled trajectories.

12 · Too noisy to be useful

There is a catch, and it is a big one. The estimator we just derived is unbiased, which sounds like high praise, but in practice it is nearly useless: its **variance is enormous**. Here is why.

The return $R(\tau)$ is a sum over the whole trajectory. Its scale depends on the game, the length, the discount. Worse, it is almost always positive, so every observed trajectory says "do more of what you just did" — including trajectories that were merely average. All trajectories push the same direction, differing only in how hard they push. That is a terrible learning signal.

The fix is embarrassingly simple. Subtract a number from the return:

$$R_t \;\longmapsto\; R_t - b(s_t)$$

where $b(s_t)$ is called a **baseline**, and R_t is the return from step t onward (the *reward-to-go*, since reward that arrived before step t cannot have been caused by the action at step t).

Subtracting a baseline leaves the gradient unbiased. That is worth proving, because the proof is three lines and it explains *why* the baseline is allowed to depend on the state but not on the action:

$$\begin{aligned}
\mathbb{E}_{a\sim\pi}\big[\nabla_\theta \log \pi_\theta(a \mid s)\, b(s)\big] &= b(s) \sum_a \nabla_\theta \pi_\theta(a \mid s) && (b \text{ pulls out}) \\
&= b(s)\, \nabla_\theta \underbrace{\sum_a \pi_\theta(a \mid s)}_{=1} && (\text{swap sum and gradient}) \\
&= b(s) \cdot 0 = 0 && (\text{gradient of a constant})
\end{aligned}$$

The probabilities always sum to one. Whatever you do to θ, they still sum to one. So the gradient of that sum is zero, and any quantity that does not depend on the action gets multiplied by zero in expectation. The baseline changes the *variance* and nothing else.

But a constant does nothing at all

Now a subtlety that catches almost everyone, and that the lecture gets wrong.

You might think: subtracting *any* number reduces variance, so let us subtract the mean. No. The variance of a random variable is unchanged by adding or subtracting a constant:

$$\operatorname{Var}(X - c) \;=\; \operatorname{Var}(X) \qquad \text{for any constant } c$$

So a constant baseline is a no-op. Variance falls only when the baseline is *correlated with the return* — when it tracks the thing you are subtracting from, and so removes part of its spread. Formally:

$$\operatorname{Var}(X - b) \;=\; \operatorname{Var}(X) - 2\operatorname{Cov}(X, b) + \operatorname{Var}(b)$$

You want $\operatorname{Cov}(X, b)$ large and positive. In practice b is chosen to be a guess at the return — which brings us to the next chapter.

> **A NOTE ON THE NOTES**
>
> This is one of two outright errors in the source. At 24:57 the lecture says: suppose you take a high-variance estimator, subtract a *constant*, and the variance "would simply change." It would not. $\operatorname{Var}(X - c) = \operatorname{Var}(X)$ exactly. Variance reduction requires a state-dependent baseline that correlates with the return. The notes on page 132 repeat the claim without the constant, so they are not wrong — but they never state the condition that makes it true either.

QUESTION 12.1

Why is subtracting a baseline still unbiased? Give the one-line reason.

ANSWER

Because $\mathbb{E}_a[\nabla_\theta \log \pi_\theta(a \mid s) b(s)] = b(s) \nabla_\theta \sum_a \pi_\theta(a \mid s) = b(s) \nabla_\theta 1 = 0$ — provided b does not depend on the action.

QUESTION 12.2

Does subtracting a *constant* baseline reduce variance?

ANSWER

No. $\mathrm{Var}(X - c) = \mathrm{Var}(X)$. Variance falls only if the baseline correlates with the return — so it must depend on the state, not be a constant.

13 · Did this action help?

What should the baseline be? The natural guess is: the return you would expect anyway from this state, before you know which action was taken. That object has a name.

$$V^\pi(s) \;=\; \mathbb{E}_\pi\big[R_t \mid s_t = s\big]$$

This is the **value function**. It answers: starting here, and then behaving according to my policy, how much reward do I expect to collect? Note that it is a function of the state only. It does not know what you are about to do.

Its sibling conditions on the action as well:

$$Q^\pi(s, a) \;=\; \mathbb{E}_\pi\big[R_t \mid s_t = s,\, a_t = a\big]$$

the **action-value**, or Q-function. Same question, but now you also say which move you are making.

The difference between them is one of the most useful quantities in the subject:

$$A^\pi(s, a) \;=\; Q^\pi(s, a) - V^\pi(s)$$

called the **advantage**. And the name is exactly right. The Q term asks "how much do I get if I do this here?" The V term asks "how much do I get here regardless?" The difference is what *this action specifically* contributed. The professor puts it as: does taking this action lead to a better reward or not?

Now the small miracle. You would think computing the advantage requires computing both Q and V. It does not. Take the observed reward-to-go R_t, which is one sample of what actually happened, and subtract the value:

$$\begin{aligned}\mathbb{E}\big[R_t - V^\pi(s_t) \,\big|\, s_t{=}s,\, a_t{=}a\big] &= \underbrace{\mathbb{E}\big[R_t \mid s_t{=}s, a_t{=}a\big]}_{Q^\pi(s,a)} - V^\pi(s) \qquad \text{(since } V \text{ is constant given } s\text{)} \\ &= Q^\pi(s,a) - V^\pi(s) \;=\; A^\pi(s,a) \quad \text{(definition)}\end{aligned}$$

So $R_t - V(s_t)$ is an unbiased *estimate* of the advantage, from a single sample. That is all we ever needed: drop the expectation from an unbiased estimator and you are left with something you can compute from one rollout.

QUESTION 13.1

What is the difference between $V^\pi(s)$ and $Q^\pi(s,a)$, in words?

ANSWER

V is the expected return from a state, *not knowing* which action will be taken (averaging over the policy). Q is the expected return from a state *given* a particular action. So Q is V plus the effect of that action.

QUESTION 13.2

Why is $R_t - V(s_t)$ an estimate of the advantage, and why does it need only one sample?

ANSWER

Because $\mathbb{E}[R_t \mid s, a] = Q(s,a)$ and $V(s)$ is constant given s, so the expectation of the difference is $Q - V = A$. Dropping the expectation from an unbiased estimator leaves a single-sample estimator.

14 · The equation on every page

Put the last three chapters together. Use the advantage as the baseline. That is the equation that appears, in some clothing, in essentially every paper about aligning language models:

$$\boxed{\nabla_\theta J(\theta) \;\approx\; \mathbb{E}_{\tau\sim\pi_\theta}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\; \hat{A}^\pi(s_t, a_t)\Big]}$$

Note the $\approx$ and not $=$. The left side is exact; the right side uses a one-sample estimate of the advantage rather than its expectation. This is an estimator, not an identity, and the professor is careful to say so.

Read it as an instruction. For each step of each trajectory: compute the log-probability of the action you took, take its gradient, multiply by how much better than expected that action turned out to be, and average. Actions that beat the baseline get their probability pushed up; actions that fell short get pushed down. Repeat a few million times.

QUESTION 14.1

Recite the key equation, and say in one sentence what it instructs you to do.

ANSWER

$\nabla_\theta J \approx \mathbb{E}[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\hat{A}_t]$. Push up the log-probability of actions whose advantage was positive and down those whose advantage was negative, weighted by the size of the advantage.

PART III — PUTTING A LANGUAGE MODEL INSIDE THE MACHINE

15 · A prompt is a state

Everything so far has been about games. Now we make the substitution that the whole subject exists for, and it is simpler than you might expect.

Take the transformer. Its output at each position is a distribution over the vocabulary. Take the prompt. Treat it as the initial state.

$$
\begin{aligned}
s_0 &= \text{the prompt} && \leftarrow \text{ this is } \rho_0\text{, the prompt distribution} \\
a_0 &= \text{the first token emitted} && \text{an action} \\
s_1 &= [\, s_0,\ a_0 \,] && \text{the prompt with that token appended} \\
a_1 &= \text{the second token} && \text{the next action} \\
&\ \ \vdots
\end{aligned}
$$

That is it. The policy *is* the language model; the action space is the vocabulary; the state is the entire text so far. Nothing new is trained to make this work. The professor makes the point twice, because it is easy to over-complicate: the transformer that gives you a distribution over tokens *is* the policy, and updating the policy is nothing other than changing the transformer's weights.

Two consequences are worth noticing.

First, the action space is enormous. In the Atari game it had a handful of elements. Here it has fifty thousand or more — one action per vocabulary item. That makes this a very unusual reinforcement learning problem.

Second, where does the reward land? A preference is a judgement about a *whole response*, so the reward arrives once, at the end. Every intermediate step is worth zero:

$$r(s_t, a_t) = 0 \ \ \forall\, t < T, \qquad r(s_T, a_T) = r_\phi(x, y) \ \in \ \mathbb{R}$$

This is called a *sparse, terminal* reward, and it explains a great deal about how these systems are built. The learning signal for a forty-token response is a single number,

arriving at token forty. Which is why, in practice, the discount factor is set to $\gamma = 1$ — episodes are short and bounded by the context window, so future reward is not something you need to deflate.

QUESTION 15.1

In the language-model MDP, what are s_t, a_t, and $\mathcal{A}$?

ANSWER

s_t is the entire text so far (prompt plus generated tokens), a_t is the next token, and $\mathcal{A} = \mathcal{V}$ is the whole vocabulary — tens of thousands of actions.

QUESTION 15.2

When is the reward nonzero, and why does that matter?

ANSWER

Only at the final step, as one number scoring the whole response. It is a sparse terminal reward, which is why the advantage must be estimated from the episode's outcome and why some architectures add a learned value function to help.

16 · Stale samples, and the ratio that fixes them

Here is a practical annoyance that turns out to have a beautiful fix.

Look again at the update. The expectation is over trajectories drawn from π_θ — the very policy we are changing. This is called **on-policy** learning, and it is brutally expensive: the moment θ moves, the samples you just collected came from the wrong distribution. Throw them away and generate again.

For a language model, "generate again" means running full inference — sampling complete responses to prompts — which dominates the cost of training. Wouldn't it be better to reuse a batch for several gradient steps?

The tool for that is older than any of this, and it is called **importance sampling**. Suppose you want an expectation under a distribution p, but you only have samples from q. Multiply and divide by q:

$$\mathbb{E}_{x\sim p}\big[f(x)\big] = \int q(x) \underbrace{\frac{p(x)}{q(x)}}_{\text{importance weight}} f(x)\,dx = \mathbb{E}_{x\sim q}\Big[\frac{p(x)}{q(x)} f(x)\Big]$$

Samples from the wrong distribution can be corrected by weighting each one by how much more (or less) likely it was under the distribution you actually wanted. The price is that you must be able to evaluate the ratio p/q.

Now the beautiful part. For a language model you *can* — trivially. You generated that token under the old checkpoint, and the log-probability it assigned was written to the rollout buffer. The current model assigns some log-probability too. The ratio is two numbers and a subtraction:

$$r_t(\theta) \;=\; \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

This is only possible because the policy is an explicit probability model. You could never do this with an agent that only knows Q-values. So we can write the gradient as an expectation over the *old* policy, and take several gradient steps before regenerating:

$$\underbrace{\mathbb{E}_{\pi_{\text{old}}}\left[r_t(\theta)\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)\,\hat{A}_t\right]}_{\text{what we want}} \qquad \text{and} \qquad \underbrace{L(\theta) = \mathbb{E}_{\pi_{\text{old}}}\left[r_t(\theta)\,\hat{A}_t\right]}_{\text{what we optimise}}$$

These are not the same function. One is *ratio times grad-log*; the other is *grad-ratio*. They coincide because

$$\nabla_\theta r_t(\theta) = \nabla_\theta \frac{\pi_\theta}{\pi_{\text{old}}} = \frac{\pi_\theta}{\pi_{\text{old}}} \nabla_\theta \log \pi_\theta = r_t(\theta)\,\nabla_\theta \log \pi_\theta$$

so that at $\theta = \theta_{\text{old}}$, where the ratio is exactly one,

$$\nabla_\theta L(\theta)\big|_{\theta=\theta_{\text{old}}} = \nabla_\theta J(\theta)\big|_{\theta=\theta_{\text{old}}}$$

Note the qualifier, which the notes state and which matters. The surrogacy holds *at* the old parameters. It is a first-order approximation to the true objective around the current policy, not an identity everywhere. The further you step, the more you are trusting a Taylor expansion.

QUESTION 16.1

What does “on-policy” mean, and what specifically does it cost you?

ANSWER

The expectation is over trajectories from the policy currently being optimised. So every update needs freshly generated rollouts — for an LLM, full inference on the model being trained, which is the dominant cost.

QUESTION 16.2

Why is a transformer an unusually good place to use importance sampling?

ANSWER

Because it is an explicit likelihood model: the log-probability of the sampled token under the old policy was recorded, and the current model can recompute it in one forward pass. The importance ratio $\pi_\theta/\pi_{\text{old}}$ is therefore cheap and exact.

17 · When the ratio explodes

There is a failure mode, and understanding it explains the most distinctive piece of engineering in the whole subject.

The surrogate is only valid near θ_{old}. Push θ far enough and the ratio $r_t(\theta)$ — a ratio of probabilities, recall, which can be arbitrarily large — multiplies your advantage estimate by something enormous. The objective explodes, the gradient estimate becomes wildly noisy, and training destabilises. The lecture describes exactly this: if the new policy deviates too much from the old one, the objective explodes and gives high-variance estimates.

The principled fix is to forbid the policy from moving too far. This is called a **trust region**: maximise the surrogate, but only within a neighbourhood of the old policy, measured by KL divergence.

$$\max_{\theta} \ \mathbb{E}_{\pi_{\text{old}}}\left[r_t(\theta)\hat{A}_t\right] \quad \text{subject to} \quad \mathbb{E}_{s\sim\pi_{\text{old}}}\Big[D_{\text{KL}}\big(\pi_{\text{old}}(\cdot \mid s) \,\big\|\, \pi_\theta(\cdot \mid s)\big)\Big] \le \delta$$

This is TRPO — trust region policy optimization — and the region is exactly the set of policies the surrogate can be trusted to describe.

But it is awkward to actually run. You are solving a constrained optimisation problem where the constraint is a KL divergence over a fifty-thousand-way categorical distribution, at every state. That wants a second-order solver, and it is expensive.

QUESTION 17.1

Why does the surrogate objective blow up when the policy moves too far?

ANSWER

Because the importance ratio $\pi_\theta/\pi_{\text{old}}$ is unbounded. Far from θ_{old} it multiplies the advantage by a huge factor, so the objective — and its gradient estimate — becomes enormous and high-variance.

QUESTION 17.2

What is TRPO's constraint, in words and in symbols?

ANSWER

Keep the new policy inside a trust region around the old one: maximise the surrogate subject to $\mathbb{E}_s[D_{\text{KL}}(\pi_{\text{old}}(\cdot \mid s)\|\pi_\theta(\cdot \mid s))] \le \delta$.

18 · The clip

PPO replaces the hard constraint with something almost embarrassingly crude, and it works.

Instead of measuring the KL divergence, just refuse to let the ratio leave a narrow band around one. Take the ordinary surrogate term $r_t\hat{A}_t$, and also compute the same term with the ratio clamped into $[1-\epsilon,\ 1+\epsilon]$, where ϵ is around 0.1 to 0.2. Then take the *smaller* of the two:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\ \min\Big(r_t(\theta)\,\hat{A}_t,\ \ \text{clip}\big(r_t(\theta),\,1-\epsilon,\,1+\epsilon\big)\,\hat{A}_t\Big)\Big]$$

The *min* is doing something specific, and it is easier to see by cases than by algebra. Suppose the advantage is positive — this was a good action, we want more of it. Then the objective grows as the ratio grows, and we would happily push the probability up forever. But once the ratio passes $1+\epsilon$, the clipped term is flat, and the *min* selects the flat one. There is nothing left to gain. The incentive to keep pushing is removed.

Now suppose the advantage is negative — a bad action, we want less of it. Then the objective grows as the ratio *shrinks*. If the ratio has gone above the band, the unclipped term is a large penalty, and the *min* selects it. So the gradient still flows, still pushing the ratio down where it belongs.

The asymmetry is the algorithm. When you have moved too far in the *beneficial* direction, the objective goes flat and the update stops. When you have moved too far in the *harmful* direction, the objective stays steep and the update continues. Clipping never protects a bad update. It only declines to reward a good one further.

	advantage > 0	advantage < 0
ratio inside the band	gradient flows	gradient flows
ratio above 1 + eps	**flat, stop**	**still penalised**
ratio below 1 − eps	still rewarded	**flat, stop**

The two regimes. PPO's clipped objective is flat exactly where you have already gone far enough in a direction that helps, and steep where you have gone far in a direction that hurts.

QUESTION 18.1

Write the PPO clipped objective from memory. What is ϵ roughly?

ANSWER

$L^{\text{CLIP}} = \mathbb{E}_t[\min(r_t\hat{A}_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon)\hat{A}_t)]$, with $\epsilon \approx 0.1$ to 0.2.

QUESTION 18.2

Why $\min$ rather than $\max$?

ANSWER

$\min$ is the pessimistic choice: it flattens the objective once you have moved far in a beneficial direction (so you stop), but keeps it steep when you have moved far in a harmful direction (so you keep correcting). A $\max$ would let the policy run away.

19 · What the clip does, in numbers

One worked example is worth more than the asymmetry argument, so here is a concrete one. Take a three-token vocabulary, $\epsilon = 0.1$, and suppose the old policy put probability 0.50 on token A while the current one puts 0.60.

$$r = \frac{0.60}{0.50} = 1.20, \qquad \text{clip}(1.20,\, 0.9,\, 1.1) = 1.10$$

The ratio is above the band. Now compute the objective both ways for each sign of the advantage.

ADVANTAGE	UNCLIPPED $r\hat{A}$	CLIPPED $1.1\hat{A}$	MIN	WHAT HAPPENS
$+0.8$	$1.20 \times 0.8 = 0.96$	$1.10 \times 0.8 = 0.88$	**0.88**	clipped wins, so the objective is constant — **no gradient**. We already raised a good action's probability by more than ϵ; the algorithm declines to reward that further.
-0.8	$1.20 \times (-0.8) = -0.96$	$1.10 \times (-0.8) = -0.88$	-0.96	unclipped wins, so **the gradient flows**. We wrongly raised a *bad* action's probability; the objective keeps pushing the ratio down.

Same ratio, same clip, opposite behaviour — and the sign of the advantage is the only difference. That is precisely the asymmetry, made arithmetic.

QUESTION 19.1

With $r = 1.2$, $\epsilon = 0.1$ and $\hat{A} = +0.8$, which term does the $\min$ choose, and what does that imply about the gradient?

ANSWER

The clipped term, $1.1 \times 0.8 = 0.88 < 0.96$. The objective is locally constant there, so the gradient with respect to that move is zero — no further reward for increasing the probability.

PART IV — WHERE THE NUMBER COMES FROM

20 · You cannot grade on an absolute scale

Everything so far assumed we have r. For mathematics and code we do — there is a checker. For ordinary conversation we do not, and this is where the "human feedback" in the name earns its place.

The obvious plan is to train a reward model by supervised regression. Collect triples of situation, action and score, and fit a network to predict the score. The notes propose exactly this and then immediately note the problem in one sentence: *however, obtaining such data is not easy.*

It is worth spelling out why, because it is the reason for everything in the next two chapters. Ask two people to score the same reply out of ten and you get a six and a nine. Ask them which of two replies is better and they agree far more often. Absolute scores carry the rater's private calibration — their mood, their standards, their generosity. Comparisons cancel all of that out within a single judgement.

So change the data format. Instead of scores, collect pairs:

$$\mathcal{D} = \left\{ \left(x,\, y_w,\, y_l \right) \right\}$$

where x is the prompt, y_w the response a human preferred, and y_l the one they rejected. The notes call this *preferential data*, and end a page with the question that the rest of this section answers: given preferential data, how would one build a reward model?

QUESTION 20.1

Why are pairwise preferences easier to collect than absolute scores?

ANSWER

Because a comparison cancels the rater's private calibration. Two people will disagree about whether a reply is a 6 or a 9, but agree far more reliably about which of two replies is better. Comparing is also a cheaper cognitive task than authoring or grading.

21 · Two answers, one question

We need a probabilistic model of a human choosing between two things. There is a classical one, from psychometrics, dating to 1952: the Bradley–Terry model.

The idea is almost too simple. Give each response a single number, its reward. Then say the probability that one beats the other is a softmax over those numbers:

$$P(y_w \succ y_l \mid x) = \frac{\exp\big(r(x, y_w)\big)}{\exp\big(r(x, y_w)\big) + \exp\big(r(x, y_l)\big)} \;=\; \sigma\Big(r(x, y_w) - r(x, y_l)\Big)$$

where $\sigma(t) = 1/(1 + e^{-t})$ is the logistic function. The two expressions are the same thing — divide top and bottom by the numerator and you get the sigmoid of the difference. The notes write both, one under the other, which is slightly confusing on a first read.

To fit it, maximise the likelihood of the observed comparisons:

$$\mathcal{L}_{\mathrm{RM}}(\phi) = \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}}\Big[\log \sigma\Big(r_\phi(x, y_w) - r_\phi(x, y_l)\Big)\Big]$$

Look at what this loss is actually doing. It is a logistic loss on the *difference* of rewards. When the model already ranks the winner above the loser by a wide margin, σ is near one, the log is near zero, and there is almost no gradient — the pair is learned. When the model has them backwards, σ is near zero, the log goes to minus infinity, and the gradient is large. The loss spends its effort on the pairs the model currently gets wrong.

QUESTION 21.1

Write the Bradley–Terry model and its training objective.

ANSWER

$P(y_w \succ y_l \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$, fitted by maximising $\mathbb{E}_{\mathcal{D}}[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$.

QUESTION 21.2

Which pairs does the reward-model loss concentrate on, and why?

ANSWER

The ones the model currently ranks wrongly. A correctly-ranked pair gives $\sigma \approx 1$ and $\log \sigma \approx 0$, so almost no gradient; a reversed pair gives a large gradient.

22 · A reward defined only by its differences

Now a property of Bradley–Terry that neither source mentions, and which you need in order to understand the next part.

Add a constant to the reward. Every response gets $+100$. What happens to the model? Nothing:

$$\big(r_\phi + c\big)(x, y_w) - \big(r_\phi + c\big)(x, y_l) = \underbrace{r_\phi(x, y_w) - r_\phi(x, y_l)}_{\text{unchanged}}$$

The constant cancels in the difference. Which means the likelihood cannot tell r from $r + 100$ — indeed from r plus *any function of the prompt alone,* since both responses share

the same x. Only differences between responses to the same prompt are **identifiable**.

This is not a technicality. It is the crack in the wall through which the next part of the essay walks. If a reward model is only defined up to a per-prompt constant, then a per-prompt constant is *unobservable* — and anything that depends on it can be cancelled out of the mathematics entirely.

Two more things BT does not model, for completeness. It cannot represent a tie: it insists one answer wins. And it assumes human preference really is a logistic function of a scalar difference — an assumption, not a theorem. If the features a human cares about cannot be summarised by one number per response, the reward model is misspecified, and the optimiser downstream will faithfully optimise the misspecification.

QUESTION 22.1

Why is a Bradley–Terry reward only defined up to a per-prompt constant?

ANSWER

Because the model depends only on $r(x, y_w) - r(x, y_l)$, and both responses share the same x. Adding any function of x alone — in particular a constant — cancels in the difference and leaves the likelihood unchanged. So only differences between responses to the same prompt are identifiable.

QUESTION 22.2

Name two things Bradley–Terry fails to represent.

ANSWER

Ties (it forces a strict winner), and any preference that cannot be expressed as a logistic function of a single scalar difference between two responses.

23 · Goodhart, with a gradient

We now have a reward model r_ϕ. Freeze it. Run PPO against it.

And here the whole scheme develops a disease, which is worth understanding before the cure. Recall what r_ϕ is: a network fitted to a finite pile of human comparisons. It is a *proxy* for human preference, accurate on the kind of text people actually wrote in the data.

PPO is an optimiser with no other objective. It does not know that r_ϕ is a proxy. Give it a few thousand steps and it will find inputs where r_ϕ is enormous and human

preference is not: flattery, length, confident-sounding confabulation, the tone of a helpful expert with nothing behind it.

This is Goodhart's law — *when a measure becomes a target, it ceases to be a good measure* — implemented as gradient descent. The optimiser does not break the reward model; it finds the parts of the input space where the reward model is merely wrong, and lives there.

The cure is a leash. Do not just maximise the reward; maximise the reward minus how far you have drifted from the model you started with:

$$\max_{\theta} \; \mathbb{E}\big[r_\phi(x, y)\big] - \beta \, D_{\mathrm{KL}}\big(\pi_\theta(\cdot \mid x) \,\big\|\, \pi_{\mathrm{ref}}(\cdot \mid x)\big)$$

where π_{ref} is the frozen supervised model, and β says how much drift you tolerate. Small β and the model chases the proxy into degeneracy; large β and it never moves at all. Tuning β is most of the practical art of RLHF.

> A NOTE ON THE NOTES
>
> This KL term is the second real problem in the source. Page 141 writes the full PPO loss — clipped objective, value regression, entropy bonus — and omits the reference-policy KL entirely. Then page 144, motivating DPO, silently assumes it is there. The notes are internally inconsistent on the one term that makes RLHF safe.

QUESTION 23.1

What is reward hacking, in one sentence?

ANSWER

The optimiser finds inputs that score highly under the learned reward model while being worse by actual human preference — it exploits the proxy's errors rather than the thing the proxy was meant to measure.

QUESTION 23.2

What is the standard mitigation, and what does its strength parameter trade off?

ANSWER

A KL penalty to the frozen reference policy, $-\beta D_{\mathrm{KL}}(\pi_\theta \| \pi_{\mathrm{ref}})$. Small β: chases the reward and hacks it. Large β: stays near the reference and barely improves.

PART V — THE CANCELLATION

24 · An objective with a closed-form answer

Now we arrive at the part that, the first time you see it, looks like a magic trick. It is not. It is one algebraic observation, made at exactly the right moment.

Here is the objective from the last chapter, stripped of the reward model notation:

$$\max_{\theta} \; \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\big[r(x,y)\big] - \beta\, D_{\mathrm{KL}}\big(\pi_\theta(\cdot \mid x) \,\big\|\, \pi_{\mathrm{ref}}(\cdot \mid x)\big)$$

It looks like something you would have to optimise numerically, with PPO. But it has a closed-form solution, and we can find it. The objective separates over prompts, so fix one x and maximise over the distribution $\pi(\cdot \mid x)$ directly, subject to the probabilities summing to one. With a Lagrange multiplier μ:

$$\mathcal{L} = \sum_y \pi(y)\Big[r(y) - \beta \log \frac{\pi(y)}{\pi_{\mathrm{ref}}(y)}\Big] + \mu\Big(\sum_y \pi(y) - 1\Big)$$

$$\frac{\partial \mathcal{L}}{\partial \pi(y)} = r(y) - \beta \log \frac{\pi(y)}{\pi_{\mathrm{ref}}(y)} - \beta + \mu \;=\; 0$$

Setting that derivative to zero and solving for $\pi(y)$ gives the answer.

$$\pi^\star(y \mid x) = \frac{1}{Z(x)}\, \pi_{\mathrm{ref}}(y \mid x)\, \exp\Big(\frac{1}{\beta} r(x,y)\Big)$$

Read it: start from the reference model, and tilt it exponentially towards high-reward responses. The tilt is sharp when β is small and gentle when β is large. That is exactly what a KL-regularised optimum should look like, and the KL term is precisely what makes it tractable — without it the optimum is the degenerate "put all probability on the single best response".

And $Z(x)$ is the normaliser, $Z(x) = \sum_y \pi_{\mathrm{ref}}(y \mid x) \exp\big(r(x,y)/\beta\big)$ — a sum over *every possible response*. Which is hopeless.

QUESTION 24.1

What is the closed-form optimum of the KL-regularised reward objective, and in words, what is it?

ANSWER

$\pi^\star(y \mid x) = \frac{1}{Z(x)}\pi_{\mathrm{ref}}(y \mid x)\exp(r(x,y)/\beta)$. It is the reference model re-weighted by an exponential tilt toward high reward, with β controlling how sharp the tilt is.

25 · The function that cancels

We cannot compute $Z(x)$. But watch what happens if we turn the equation around.

Take logs of the closed form and rearrange for r:

$$\log \pi^\star(y \mid x) = \log \pi_{\mathrm{ref}}(y \mid x) - \log Z(x) + \tfrac{1}{\beta} r(x,y)$$

$$\Longrightarrow \quad r(x,y) = \beta \log \frac{\pi^\star(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)} + \beta \log Z(x)$$

This says something startling. The optimal reward *is* the log-ratio of the optimal policy to the reference policy, plus a per-prompt constant. The reward is not something separate that has to be learned. It is already latent inside the policy.

Now put it back into Bradley–Terry, which needs a difference of rewards for two responses to the same prompt:

$$r(x, y_w) - r(x, y_l) = \beta \log \frac{\pi^\star(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^\star(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \underbrace{+\beta \log Z(x) - \beta \log Z(x)}_{=\,0}$$

And there it is. The intractable sum over every possible response appears twice, with the same value, and subtracts out.

Why *exactly* the same value? Because $Z(x)$ is a function of the prompt alone, and both responses were answers to the **same prompt**. That is the whole trick. If the two responses had been to different prompts, Z would not cancel, and this algorithm would not exist. An entire family of modern training methods rests on the observation that you compare two answers to one question.

QUESTION 25.1

Why does the partition function $Z(x)$ cancel in DPO? Be precise.

ANSWER

Because $Z(x)$ depends only on the prompt x, and the preferred and rejected responses are answers to the *same* prompt. It therefore appears twice in the reward difference with identical value and subtracts to zero.

QUESTION 25.2

What would break if the two responses came from different prompts?

ANSWER

$Z(x_w) \neq Z(x_l)$ in general, so the terms would not cancel, and you would be back to needing to compute an intractable normaliser. DPO's derivation requires a shared prompt.

26 · DPO

So we never need the reward model. Substitute the policy for the reward, take the negative log-likelihood, and you have a loss you can compute with two forward passes:

$$\boxed{\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}\Big[\log \sigma\Big(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\Big)\Big]}$$

Look at how similar this is to the reward-model loss of chapter 21. It is the same $-\log\sigma(\text{difference})$ — with the learned reward r_ϕ replaced by the quantity

$$\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}$$

which people call the *implicit reward*. That is the whole content of DPO: it is Bradley–Terry reward modelling in which the policy is its own reward function.

So the common claim that DPO is "reward-free" is imprecise. The reward is not absent; it is implicit, and it is a log-ratio. What disappears is the separate reward *network*, the value head, and the rollouts.

There is a pleasing practical consequence. At initialisation, when $\pi_\theta = \pi_{\text{ref}}$, every log-ratio is zero, so the difference is zero, so $\sigma(0) = \frac{1}{2}$, so the loss is exactly $\ln 2 \approx 0.693$. If your first training step does not report roughly 0.693, your reference model is not a frozen copy of your policy — a classic bug, caught by arithmetic.

QUESTION 26.1

Write the DPO loss and define the implicit reward.

ANSWER

$\mathcal{L}_{\text{DPO}} = -\mathbb{E}[\log\sigma(\hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l))]$ with $\hat{r}_\theta(x, y) = \beta \log(\pi_\theta(y \mid x)/\pi_{\text{ref}}(y \mid x))$.

QUESTION 26.2

Is DPO reward-free? What value does the loss take at initialisation, and why?

ANSWER

Not quite — the reward is implicit, $\beta \log(\pi_\theta/\pi_{\text{ref}})$; what is gone is the reward *network* and the RL loop. At init the loss is exactly $\ln 2 \approx 0.693$ because every log-ratio is zero, so the sigmoid is $\frac{1}{2}$.

27 · Removing the value function

One last thread, which the lecture mentions in passing and the notes omit entirely.

Go back to the game. The advantage needed a baseline, and the natural baseline was the value function — a second network, trained by regression, purely to estimate what you would have got anyway.

If the reward is verifiable, you can do something much cheaper. Sample a *group* of G answers to the same prompt. Score them all with the checker. Now use the group's own average as the baseline, and its spread to normalise:

$$\mathcal{A}_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}$$

No value network. No reward model either, since the verifier *is* the reward. The group is the baseline: if one answer scores higher than its siblings, that is evidence the answer was good, and the normalisation makes the scale irrelevant. This is GRPO, and it is the algorithm behind most modern training on mathematics and code.

Notice what made it possible. It required a *checkable* reward — chapter 4 — and a baseline that is action-independent but correlated with the return — chapter 12. Both ideas were established long before this algorithm appeared.

QUESTION 27.1

In GRPO, what replaces the value function as the baseline, and what does it require?

ANSWER

The mean reward of a group of G sampled answers to the same prompt, normalised by the group's standard deviation. It requires a *verifiable* reward, since there is no learned reward model.

PART VI — WHAT WE GLOSSED OVER

28 · Five places the notes are wrong

We promised at the start to be honest about the source. Here is the account.

One. Page 130 writes "policy updating using gradient descent" immediately beside $\theta \leftarrow \theta + \alpha \nabla_\theta J$. The plus sign means *ascent*. The words are wrong, the formula is right. (The lecture self-corrects out loud, which is charming: "gradient descent or gradient ascent rather".)

Two. Page 130 writes the return as $\sum_t r(s_t, a_t)$ without the γ^t, one page after introducing γ. The lecture writes it correctly.

Three. At 24:57 the lecture says that subtracting a *constant* baseline changes the variance. It does not: $\text{Var}(X - c) = \text{Var}(X)$. The variance falls only for a baseline that correlates with the return.

Four. Page 141's full PPO loss omits the reference-policy KL, which page 144 then assumes exists. The notes contradict themselves on the one term that prevents reward hacking.

Five. Page 143 gives Bradley–Terry as a likelihood and stops. It never states that only reward *differences* are identifiable, which is the entire basis of the next section's derivation. Reading the notes alone, the DPO cancellation looks like magic rather than algebra.

None of this makes the notes bad. They are handwritten lecture notes, and they compress. But a compressed source teaches you to reproduce symbols, and an uncompressed one teaches you why the symbols are there.

QUESTION 28.1

Name the two outright mathematical errors in the source.

ANSWER

(i) The claim that subtracting a *constant* baseline reduces variance — it does not, $\mathrm{Var}(X - c) = \mathrm{Var}(X)$. (ii) Writing the return without γ^t after introducing γ. (The "grad descent" label is a sign/word mismatch rather than a mathematical error.)

29 · What this essay does not tell you

Three honest limits, so you know where the map ends.

Scale. Everything here was described at the level of one gradient step. Nothing was said about how you batch rollouts, how you manage four models in memory, or how you keep a distributed training run stable for weeks. That is most of the engineering, and none of the mathematics.

Evaluation. We asserted that a reward model can be wrong and that DPO can displace likelihood onto unwanted responses. We did not say how to detect it. In practice this is the hard part: deciding whether a model that scores well has become better, or has merely become better at the thing you measured.

Alternatives. There are many objectives between PPO and DPO — IPO, KTO, ORPO, and others — each trading stability, data requirements and theoretical guarantees differently. This essay described the two that the course covers, which are also the two you will meet first.

And one thing to keep: everything in this essay came from two moves. First, notice that *only one factor of the trajectory distribution contains the parameters*. Second, notice that *a constant which depends only on the prompt cannot affect a comparison between two responses to that prompt*. The first gives you the policy gradient. The second gives you DPO. If you

remember nothing else, remember those two observations and you can reconstruct the rest.

QUESTION 29.1

State the two structural observations that the whole essay rests on.

ANSWER

(i) In $p_\theta(\tau) = \rho_0 \prod_t \pi_\theta\, p$, only the policy factor depends on θ — which yields the policy gradient theorem. (ii) A quantity depending only on the prompt cancels in a comparison between responses to that prompt — which yields the DPO cancellation.

Appendix · Every question in this essay

Every card you met is listed below. Try them cold: cover the right-hand column, or better, use the companion file (flashcards.tsv), which contains each question with its answer and imports directly into any spaced-repetition system. Answers are deliberately *not* printed here — a review list with the answers beside it is a reading list, not a test.

1.1 What exactly does a next-token-prediction model learn — a program, a database of answers, or something else?

1.2 Why doesn't that training objective already make the model helpful?

2.1 What is the “format” wall that stops us from just using supervised fine-tuning?

2.2 What is the “location” wall?

3.1 In one line: what is the difference between supervised learning and reinforcement learning?

4.1 What makes a reward “verifiable”, and give the two domains where this works best.

4.2 Why is an open-ended chat assistant the hard case?

5.1 In the Atari framing, what is the “state”, what is the “action”, and where does the “reward” come from?

6.1 What does the word “Markov” assert, and what does that force the state to be?

7.1 What are the two distinct reasons for the discount factor γ?

8.1 Write the trajectory distribution, and say which factors depend on θ.

8.2 Why does that factorisation matter so much?

9.1 Why is $\mathbb{E}_\tau\big[\nabla_\theta R(\tau)\big]$ useless?

9.2 In one phrase: what is being differentiated in $\nabla_\theta J$?

10.1 State the log-derivative trick and prove it in one line.

10.2 What does the trick convert the problem *into*, and why does that help?

11.1 What does the policy gradient theorem free you from needing, and why is that the difference between theory and practice here?

11.2 Name the three things you need in order to run this estimator.

12.1 Why is subtracting a baseline still unbiased? Give the one-line reason.

12.2 Does subtracting a *constant* baseline reduce variance?

13.1 What is the difference between $V^\pi(s)$ and $Q^\pi(s,a)$, in words?

13.2 Why is $R_t - V(s_t)$ an estimate of the advantage, and why does it need only one sample?

14.1 Recite the key equation, and say in one sentence what it instructs you to do.

15.1 In the language-model MDP, what are s_t, a_t, and $\mathcal{A}$?

15.2 When is the reward nonzero, and why does that matter?

16.1 What does "on-policy" mean, and what specifically does it cost you?

16.2 Why is a transformer an unusually good place to use importance sampling?

17.1 Why does the surrogate objective blow up when the policy moves too far?

17.2 What is TRPO's constraint, in words and in symbols?

18.1 Write the PPO clipped objective from memory. What is ϵ roughly?

18.2 Why min rather than max?

19.1 With $r = 1.2$, $\epsilon = 0.1$ and $\hat{A} = +0.8$, which term does the min choose, and what does that imply about the gradient?

20.1 Why are pairwise preferences easier to collect than absolute scores?

21.1 Write the Bradley–Terry model and its training objective.

21.2 Which pairs does the reward-model loss concentrate on, and why?

22.1 Why is a Bradley–Terry reward only defined up to a per-prompt constant?

22.2 Name two things Bradley–Terry fails to represent.

23.1 What is reward hacking, in one sentence?

23.2 What is the standard mitigation, and what does its strength parameter trade off?

24.1 What is the closed-form optimum of the KL-regularised reward objective, and in words, what is it?

25.1 Why does the partition function $Z(x)$ cancel in DPO? Be precise.

25.2 What would break if the two responses came from different prompts?

26.1 Write the DPO loss and define the implicit reward.

26.2 Is DPO reward-free? What value does the loss take at initialisation, and why?

27.1 In GRPO, what replaces the value function as the baseline, and what does it require?

28.1 Name the two outright mathematical errors in the source.

29.1 State the two structural observations that the whole essay rests on.

Built from pages 128–146 of the IIT Madras generative-AI notes and the lecture recording. Typeset with KaTeX. Source: *viz/RLHF-Curious.src.html*.